CIVICSIFT AI

WEEK 2

DATA COLLECTION, PREPROCESSING AND FEATURE ENGINEERING

An Evidence-Aware Multilingual NLP Framework for Citizen Complaint Triage and Incident Discovery

Submitted by: Riya Desai
Internship / Organisation: YuvaIntern
Project: CivicSift AI
Report: Week 2 – Methodology Document

Report Overview

This report documents the proposed Week 2 methodology for collecting, preparing and transforming text data for CivicSift AI. The project is a multilingual, human-assisted NLP framework intended to organise citizen complaints about public services. Its initial language scope is English, Hindi and Kannada, with controlled testing of Romanised and code-mixed messages.

The report turns the Week 1 plan into an operational data-preparation strategy. It describes candidate public data sources, a reproducible collection process, a proposed record schema, privacy and licence checks, annotation guidance, text cleaning, tokenisation, stop-word handling, stemming and lemmatisation, feature extraction, quality checks, and pseudocode. It also explains limitations and safeguards.

Important status note: this is a methodology and preparation plan. No claim is made that a dataset has already been downloaded, annotated, cleaned, or evaluated. Dataset size and availability targets remain provisional until the sources and permissions are checked.

Contents

- 1. Project context and Week 2 objectives
- 2. Dataset strategy and collection plan
- 3. Data schema, labels and annotation
- 4. Data quality, privacy and governance
- 5. Text preprocessing pipeline
- 6. Feature engineering methods
- 7. Recommended feature strategy for CivicSift AI
- 8. End-to-end workflow and pseudocode
- 9. Pitfalls, mitigation and quality assurance
- 10. Week 2 execution plan and deliverables
- 11. Conclusion
- References and appendices

1. Project Context and Week 2 Objectives

1.1 Project background

CivicSift AI is designed to help authorised reviewers organise free-text citizen complaints concerning issues such as water supply, roads, sanitation, drainage, electricity and public-safety hazards. A complaint may be written in English, Hindi or Kannada, may contain informal spelling, and may switch between languages or scripts. Two people may describe the same issue using different words.

The proposed system has three related NLP functions: (i) suggest a complaint category, (ii) surface possible urgency indicators with visible text evidence, and (iii) retrieve semantically similar complaints that may relate to the same incident. These are decision-support functions; a human reviewer remains responsible for confirming the final record.

1.2 Week 2 objectives

- Identify lawful, relevant public data sources and define a transparent collection procedure.
- Create a consistent data structure for complaint text, language/script, labels, provenance and duplicate groups.
- Plan privacy-aware cleaning while preserving information that affects meaning, such as negation, numbers, dates and urgency cues.
- Define preprocessing methods for English, Hindi, Kannada, transliteration and code-mixed messages.
- Compare TF-IDF, character n-grams and multilingual embeddings for the project's downstream tasks.
- Specify reproducible pseudocode, quality checks, risks and outputs for the next implementation stage.

1.3 Scope and assumptions

Item	Week 2 working decision
Project languages	English, Hindi and Kannada; include a small controlled set of Romanised and code-mixed examples.
Initial data target	Approximately 1,000–2,000 examples as a proof-of-concept target, subject to source availability and annotation capacity.
Main task	Complaint category classification; urgency and duplicate discovery are separate supporting tasks.
Data types	Permitted public records, synthetic examples, and consented examples where available.
Human role	Review labels, low-confidence predictions, urgency suggestions and possible duplicate matches.
Status	Proposed methodology; not an assertion that data collection or model training is complete.

2. Dataset Strategy and Collection Plan

2.1 Why dataset selection matters

The quality of an NLP system depends heavily on whether its training and evaluation examples represent the language, writing style and categories it will encounter. A dataset made only of formal English sentences would not adequately represent short, informal Indian-language complaints. Data collection must therefore consider language coverage, category coverage, source permissions, privacy and annotation reliability.

2.2 Proposed data-source strategy

A single ready-made public dataset containing balanced, labelled English-Hindi-Kannada civic complaints may not be available. The project will therefore use a carefully documented combination of sources rather than assume that one dataset satisfies every requirement.

Source type / candidate	Potential use	Limitations and checks
Official open civic-service or 311 complaint datasets (for example, municipal open-data portals)	Realistic complaint wording, service categories, timestamps and recurring issue descriptions; useful for English-language structure and category design.	Check the portal's licence, terms, fields, personal-data exposure, geographic context and whether free-text descriptions are included.
Public Indian-language NLP corpora and resources	Support language-aware preprocessing experiments, vocabulary checks and multilingual model selection.	Many are not civic-complaint datasets and may not contain complaint labels. Do not relabel unrelated text as real complaints.
Synthetic complaint examples written for the project	Controlled coverage of water, roads, sanitation, drainage, electricity and safety; create Hindi, Kannada, English, transliterated and code-mixed examples.	Clearly mark as synthetic. Do not present generated examples as real citizen reports or use them as a substitute for representative real-world evaluation.
Consented or institutionally authorised examples, if available	Improve local realism and capture actual phrasing.	Obtain documented permission, minimise personal data and follow the organisation's data-handling rules.

Recommended approach: use a relevant public civic dataset only after its terms and fields are checked; supplement it with clearly marked synthetic English, Hindi and Kannada examples for pipeline testing. Keep source type as a field so that results can be analysed separately for public, synthetic and consented data.

2.3 Dataset discovery and selection criteria

- Relevance: records should describe public-service problems or contain fields that can support a defensible mapping to the complaint taxonomy.
- Licence and terms: the intended research or internship use must be permitted. Record the exact licence or terms page and access date.
- Language: note the actual languages present. Do not infer language solely from a city or region.
- Text availability: ensure that a usable free-text description exists; category codes alone are not enough for text preprocessing.
- Metadata: retain only useful fields such as source ID, category, time period and coarse administrative area when permitted.
- Quality: inspect missing values, repeated records, boilerplate, machine-generated entries and inconsistent labels.
- Privacy: exclude or mask unnecessary names, phone numbers, email addresses, exact home addresses and other identifiers.

2.4 Collection procedure

1. Prepare a source register containing source name, provider, URL, access date, licence, permitted use, language, fields and known restrictions.
2. Read the provider's licence, terms of use and documentation before downloading. If permission is unclear, do not use the source until clarified.

3. Download through the official portal or documented API. Avoid scraping sites in ways that violate their terms or access controls.
4. Save the original file unchanged in a restricted raw-data folder. Record its filename, download date, source version and checksum where practical.
5. Create a working copy for cleaning. Never overwrite the original source file.
6. Map source fields into the CivicSift schema. Preserve the original category separately from the project's mapped category.
7. Run initial profiling: record count, missing text, language distribution, category distribution, duplicate rate and suspicious values.
8. Document every filtering or transformation decision in a data-preparation log.

2.5 Proposed complaint taxonomy

Category	Example issue types (illustrative)
Water	Water leakage, no water supply, contaminated supply, broken public tap.
Roads	Potholes, damaged road surface, unsafe road obstruction.
Sanitation	Overflowing bins, uncollected waste, public-area cleanliness.
Drainage	Blocked drains, waterlogging, overflowing sewage.
Electricity	Street-light failure, exposed wires, power-related public infrastructure fault.
Safety hazard	Fallen structures, dangerous obstruction, immediate public-space hazard.
Other / unclear	Insufficient information or issue outside the defined categories.

The taxonomy should be mutually understandable to annotators. If a complaint genuinely contains two separate issues, use a documented multi-label rule or split it only when the source record supports doing so. Do not force ambiguous records into a category; use "Other / unclear" and flag them for review.

3. Data Schema, Labels and Annotation

3.1 Proposed record structure

Each complaint should be stored as one structured record. Keep raw text and processed text in separate fields so that transformations can be audited and the original wording remains available to authorised reviewers.

Field	Description / example
record_id	Project-generated unique identifier; do not expose a source person's identity.
source_id / source_name	Dataset or portal identifier and source name.
source_type	public, synthetic or consented.
raw_text	Original complaint text, stored with restricted access.
text_for_processing	Working text after approved privacy masking and normalisation.
language_label	English, Hindi, Kannada, mixed, unknown; manually verified on a sample.
script_label	Latin, Devanagari, Kannada, mixed or unknown.
category_label	Water, Roads, Sanitation, Drainage, Electricity, Safety hazard, Other/unclear.
urgency_label	Low, Medium, High, or Insufficient information, based on written criteria.
duplicate_group_id	Shared identifier for records known to describe the same incident; blank if unknown.
location_mention	Optional text-derived mention; not a verified geographic location.
annotation_status	unreviewed, reviewed, adjudicated.
preprocessing_version	Version identifier for the transformation pipeline.

3.2 Annotation guidelines

Annotation is the process of assigning consistent labels to text examples. It is important because supervised models learn the patterns present in the labels. The project should prepare a short annotation guide before labelling the full dataset.

- Read the complete complaint before assigning a category.
- Choose the category based on the reported public-service issue, not the writer's emotion or grammar.
- Use "Other / unclear" when the text does not provide enough evidence.
- Assign urgency from explicit impact, danger and time-sensitive cues in the text. Do not use sentiment alone.
- Preserve the difference between "not working" and "working"; negation can reverse meaning.
- Mark language and script separately. A Hindi sentence written in Latin letters is Hindi language in Latin script, not English.
- Mark code-mixed text when meaningful words or phrases from more than one language occur in the same message.
- Use a duplicate-group ID only when there is evidence that messages refer to the same incident; semantic similarity alone is not proof.

3.3 Urgency labelling proposal

Label	Operational interpretation	Illustrative cue (not an automatic rule)
High	Explicit immediate danger, risk of injury, major active hazard or severe disruption requiring prompt human attention.	"Live wire hanging near the school."
Medium	A significant service failure or hazard without clear evidence of immediate danger.	"Drain has been blocked for two days."
Low	Routine service issue with no stated immediate safety impact.	"Street light is not working."
Insufficient information	The message does not contain enough information to judge urgency.	"Please solve this issue."

These definitions are provisional. They should be reviewed with the internship supervisor or a suitable domain reviewer. The system must not make emergency-response decisions automatically.

3.4 Annotation quality

Where feasible, two annotators should independently label a subset of records. Disagreements should be reviewed against the written guidelines and resolved by an adjudicator. Report the number of double-labelled records and the agreement method used. If only one annotator is available, perform a delayed re-annotation of a sample and document the limitation.

4. Data Quality, Privacy and Governance

4.1 Initial data inspection

Before text cleaning, inspect the dataset to understand its structure and identify defects. A profile should include total records, empty text, duplicate IDs, repeated text, language/script counts, category counts, unusual text lengths and missing metadata.

Check	What to inspect	Action
Missing text	Null, empty or whitespace-only complaint fields.	Exclude from text modelling or retain separately as incomplete; record the count.
Exact duplicates	Identical text and/or repeated source IDs.	Remove accidental duplicate copies from model splits; retain source lineage.
Near duplicates	Minor spelling or punctuation changes in otherwise repeated reports.	Flag for review and group related incident records before splitting.
Invalid characters	Broken encoding, replacement symbols or corrupted bytes.	Re-read using the correct encoding; do not silently delete unknown characters.
Class imbalance	Large differences in category counts.	Report distribution; use stratified splitting and consider class weights.
Language imbalance	Some languages or scripts have very few examples.	Report per-language coverage and avoid broad performance claims.
Suspicious content	Personal identifiers, spam, advertisements or irrelevant text.	Mask, quarantine or exclude according to a documented rule.

4.2 Privacy-first handling

- Collect only fields required for the NLP task.
- Mask phone numbers, email addresses, personal names and exact addresses when they are not necessary.
- Keep any raw-to-masked mapping, if one is legitimately required, separately with strict access controls; preferably do not create such a mapping.
- Use synthetic examples in public screenshots, demos and report illustrations.
- Restrict access to raw data and avoid uploading confidential complaint records to public tools or services.
- Do not use sensitive personal characteristics to categorise, rank or prioritise complaints.
- Document retention, deletion and access arrangements according to the data provider’s rules and the internship organisation’s policy.

4.3 Data leakage prevention

A particularly important risk is allowing nearly identical complaints from one incident to appear in both training and test sets. This can make model performance appear better than it really is. Where duplicate-group or incident identifiers exist, split by group rather than randomly by individual row. Fit vocabulary, IDF weights, feature selection and other learned preprocessing steps on the training data only; then apply the fitted transformation to validation and test data.

5. Text Preprocessing Pipeline

Preprocessing converts inconsistent raw messages into a representation that NLP algorithms can process. The aim is not to make every sentence look identical. It is to remove irrelevant noise while preserving words and symbols that affect the complaint's meaning.

5.1 Recommended order of operations

```
Raw complaint
    |
    v
Source / licence / privacy checks
    |
    v
Missing-value and duplicate inspection
    |
    v
Personal-identifier masking
    |
    v
Unicode and whitespace normalisation
    |
    v
Language + script identification
    |
    v
Careful spelling / transliteration handling
    |
    v
Tokenisation (language-aware where possible)
    |
    v
Optional stop-word handling
    |
    v
Optional stemming / lemmatisation
    |
    v
Feature extraction and train-only fitting
```

5.2 Missing values and duplicate handling

Rows with no complaint text cannot contribute to text-based classification. They should be counted and excluded from the modelling text field, while their exclusion is recorded. Exact duplicates can be identified using normalised text hashes. Near-duplicates require more care: punctuation or spelling changes may represent repeated reports, but similar wording can also describe different events.

5.3 Privacy masking

Before modelling, identify patterns such as phone numbers, email addresses and long numeric identifiers. Replace them with neutral placeholders such as [PHONE] or [EMAIL] rather than deleting the surrounding sentence. Names and addresses may require named-entity recognition or manual review; automated detection is imperfect, especially for Indian names and mixed-language text.

5.4 Unicode and text normalisation

Unicode normalisation helps ensure that visually identical characters are represented consistently. NFC is a reasonable initial choice for standardising composed character sequences. Trim leading and trailing spaces, collapse repeated whitespace and standardise line breaks. Preserve the original text and store the normalised version separately.

Do not indiscriminately remove punctuation, digits or symbols. Numbers may indicate duration (“3 days”), quantity, house/building references or incident details. Exclamation marks and repeated characters may sometimes carry emphasis. Normalise only when the task benefits and the original remains available.

5.5 Language and script identification

Language and script are different properties. Hindi may be written in Devanagari or Roman letters; Kannada is commonly written in Kannada script but may also appear transliterated. A message can contain multiple scripts or languages. Use an automatic language detector only as an initial suggestion, then validate a sample manually. Short complaint messages are difficult to identify reliably.

5.6 Tokenisation

Tokenisation splits text into smaller units called tokens. Tokens may be words, punctuation marks, subwords or character sequences. For example, “Water is leaking near the school” can be tokenised into [“Water”, “is”, “leaking”, “near”, “the”, “school”].

For English, a standard word tokenizer can be used for the baseline. For Hindi and Kannada, use tokenisers compatible with the relevant scripts and language resources where available. For transformer models, use the tokenizer supplied with the selected pretrained model; do not replace it with an unrelated word tokenizer.

5.7 Stop-word removal

Stop words are frequent words that may contribute little to a particular task. Removing them can reduce feature size in a traditional bag-of-words model. However, stop-word removal must be selective. Words equivalent to “not”, “no”, “without” or their language-specific forms can reverse the meaning of a complaint and must be preserved.

Use language-specific stop-word lists and inspect them before use. Compare model performance with and without stop-word removal. For transformer embeddings, stop-word removal is generally not recommended because the model uses surrounding context and grammatical structure.

5.8 Stemming and lemmatisation

Stemming reduces a word to a shorter root-like form by removing prefixes or suffixes using rules. The output may not be a valid dictionary word. Lemmatisation converts a word to its dictionary base form (lemma), using vocabulary and grammatical information where supported.

Technique	Meaning	Strength	Limitation
Stemming	Rule-based reduction to a stem.	Fast and can reduce related word variants.	May produce non-words or remove meaningful suffixes.
Lemmatisation	Conversion to a linguistically valid base form.	Often preserves a more meaningful lexical form.	Needs language-specific tools and may be slower or unavailable for some languages.

For CivicSift AI, test stemming or lemmatisation only for the classical TF-IDF baseline and only when reliable language support exists. Do not apply an English stemmer to Hindi or Kannada. Keep an unstemmed version for comparison, and do not stem transformer input unless the model documentation specifically requires it.

5.9 Spelling variation and transliteration

Informal complaints may contain spelling errors, repeated characters, abbreviations and Romanised Indian-language words. Aggressive correction can change meaning or erase local expressions. Retain raw text; create a separate normalised field; maintain a small documented mapping for frequent, unambiguous variants; and use character n-grams to make the baseline more tolerant of spelling variation. Do not automatically translate every message into English because translation may lose local meaning.

5.10 Preserve meaning-bearing information

- Negation: preserve words such as “not”, “no”, “nahi” and their equivalents.

- Time and duration: retain “today”, “since morning”, dates and quantities such as “3 days”.
- Urgency cues: retain terms describing danger, injury, leakage, fire, exposed wires or collapse.
- Category terms: retain infrastructure words such as road, drain, water, light and garbage.
- Location mentions: preserve as text-derived mentions only; do not treat them as verified coordinates.

6. Feature Engineering Methods

Feature engineering converts prepared text into numerical representations that machine-learning algorithms can use. Different representations capture different kinds of information. CivicSift AI requires features for category classification and semantic duplicate discovery, so one representation may not be best for every component.

6.1 Bag of Words (BoW)

Bag of Words represents a document by counting how often each vocabulary term appears. Word order is ignored. For example, a small vocabulary [water, road, leak] could represent “water leak” as [1, 0, 1].

Advantages: simple, fast, interpretable and useful as a baseline. Limitations: it creates sparse vectors, ignores word order and may not recognise synonyms or related meanings across languages. A shared vocabulary across several scripts can become very large.

6.2 TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) gives higher weight to terms that are important in one document but not common across all documents. Term frequency measures how often a term occurs in a document. Inverse document frequency reduces the weight of terms that occur in many documents.

A common form is: TF-IDF(t,d) = TF(t,d) × IDF(t), where IDF(t) = log((1 + N) / (1 + df(t))) + 1. Here, N is the number of documents and df(t) is the number of documents containing term t. The exact smoothing and normalisation depend on the implementation.

Word-level TF-IDF is suitable for a transparent complaint-category baseline. It can highlight terms such as “pothole”, “leakage”, “garbage” and “streetlight”. Character-level TF-IDF can capture spelling fragments and transliteration variants, making it valuable for informal or code-mixed text.

6.3 N-grams

An n-gram is a sequence of n consecutive tokens. Unigrams contain one token, bigrams contain two, and trigrams contain three. For example, “not working” is a bigram and may be more informative than either word alone.

Word n-grams can capture short phrases such as “no water”, “street light” and “water supply”. Character n-grams can represent partial spellings and morphological patterns. They are useful when words are misspelled or written in different Romanised forms. The n-gram range should be selected using validation data rather than assumed.

6.4 Word embeddings

Word embeddings represent words as dense numerical vectors. Words used in similar contexts may have nearby vectors. Unlike basic BoW, embeddings can represent some semantic relationships even when words are not identical. Static word embeddings assign one vector per word and may struggle with ambiguity, rare words and code-mixed spellings.

6.5 Sentence embeddings

Sentence embeddings represent an entire sentence or complaint as one dense vector. A multilingual sentence-embedding model can map English, Hindi and Kannada messages into a shared semantic space, subject to model coverage and empirical testing. The project can compare complaint vectors using cosine similarity:

cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)

Sentence embeddings are especially relevant to duplicate discovery. For example, “There is a large pothole on the road” and “A deep hole is damaging the street” may have low exact word overlap but similar meaning. Similarity only retrieves candidates; it does not prove that two reports refer to the same incident.

6.6 Multilingual transformer representations

Transformer models use contextual token representations: the representation of a word can depend on the surrounding words. Multilingual models such as IndicBERT or XLM-R can be evaluated for classification across supported languages. Their tokenizer and model weights should be used as a matched pair.

Advantages include contextual information and potential cross-lingual transfer. Limitations include greater compute requirements, more complex deployment, sensitivity to domain mismatch and uncertain performance on informal local complaint text. A pretrained model should be evaluated on the project's own held-out examples.

6.7 Feature-method comparison

Method	Captures well	Weakness	CivicSift use
BoW	Term occurrence and simple counts.	No semantic similarity or word order.	Simple reference baseline.
Word TF-IDF	Important words and short phrases.	Sparse; vocabulary-dependent; weak on unseen spellings.	Primary interpretable category baseline.
Character TF-IDF	Subword patterns, spelling variants, transliteration fragments.	Less directly interpretable; can create many features.	Complement word TF-IDF for noisy/code-mixed input.
Static word embeddings	Some semantic relationships between words.	One vector per word; rare and ambiguous words are difficult.	Optional experiment if a suitable multilingual resource is available.
Sentence embeddings	Meaning at sentence level and cross-lingual similarity.	Model-dependent; similarity can confuse same-topic and same-incident reports.	Primary candidate for duplicate retrieval.
Multilingual transformer	Contextual language representation.	Higher compute and evaluation complexity.	Advanced category-classification comparison.

7. Recommended Feature Strategy for CivicSift AI

7.1 Separate features by task

CivicSift AI should not force every component to use the same feature representation. Category classification and duplicate retrieval have different requirements. A practical staged design is:

Component	Recommended representation	Reason
Category classification baseline	Word TF-IDF with unigrams and bigrams, plus a separate character TF-IDF experiment.	Fast, interpretable and suitable for Logistic Regression or Linear SVM; character features help with spelling variation.
Advanced category model	IndicBERT or XLM-R, selected after checking language coverage and compute feasibility.	Tests whether contextual multilingual representations improve over the classical baseline.
Duplicate complaint retrieval	Multilingual sentence embeddings with cosine similarity.	Compares meaning across different wording and potentially across supported languages.
Urgency support	Transparent phrase rules first; optionally a separately trained classifier if enough consistent labels exist.	Keeps evidence visible and avoids treating sentiment as urgency.
Text-derived information mentions	Language-aware rules or suitable named-entity recognition tools, followed by human confirmation.	Can surface dates, service names or location mentions without claiming they are verified facts.

7.2 Why TF-IDF is the right first baseline

TF-IDF is recommended as the first classification baseline because it is lightweight, reproducible and easy to inspect. It can be trained with limited computing resources and provides a useful point of comparison for a transformer model. Word-level features retain meaningful service terms, while character-level features can help with misspellings, transliteration and code-mixed forms.

A Linear Support Vector Machine or Logistic Regression model can be trained on the resulting sparse vectors. The two algorithms should be compared on the same group-aware data split. If a more complex multilingual model does not deliver a meaningful improvement, the simpler model may be preferable for a proof-of-concept system.

7.3 Why embeddings are useful for duplicate discovery

Duplicate discovery is not identical to category classification. Two complaints may belong to the same category but refer to different incidents. Conversely, two messages about one incident may use different words. Sentence embeddings can retrieve semantically related candidates, while time and permitted coarse location metadata may be used as additional review context. The final decision to merge reports must remain with a reviewer.

7.4 Feature selection and parameter tuning

For TF-IDF, candidate settings include word n-gram range, character n-gram range, minimum document frequency, maximum document frequency, vocabulary size and sublinear term frequency. Select settings using training and validation data only. Avoid choosing parameters after inspecting test-set outcomes.

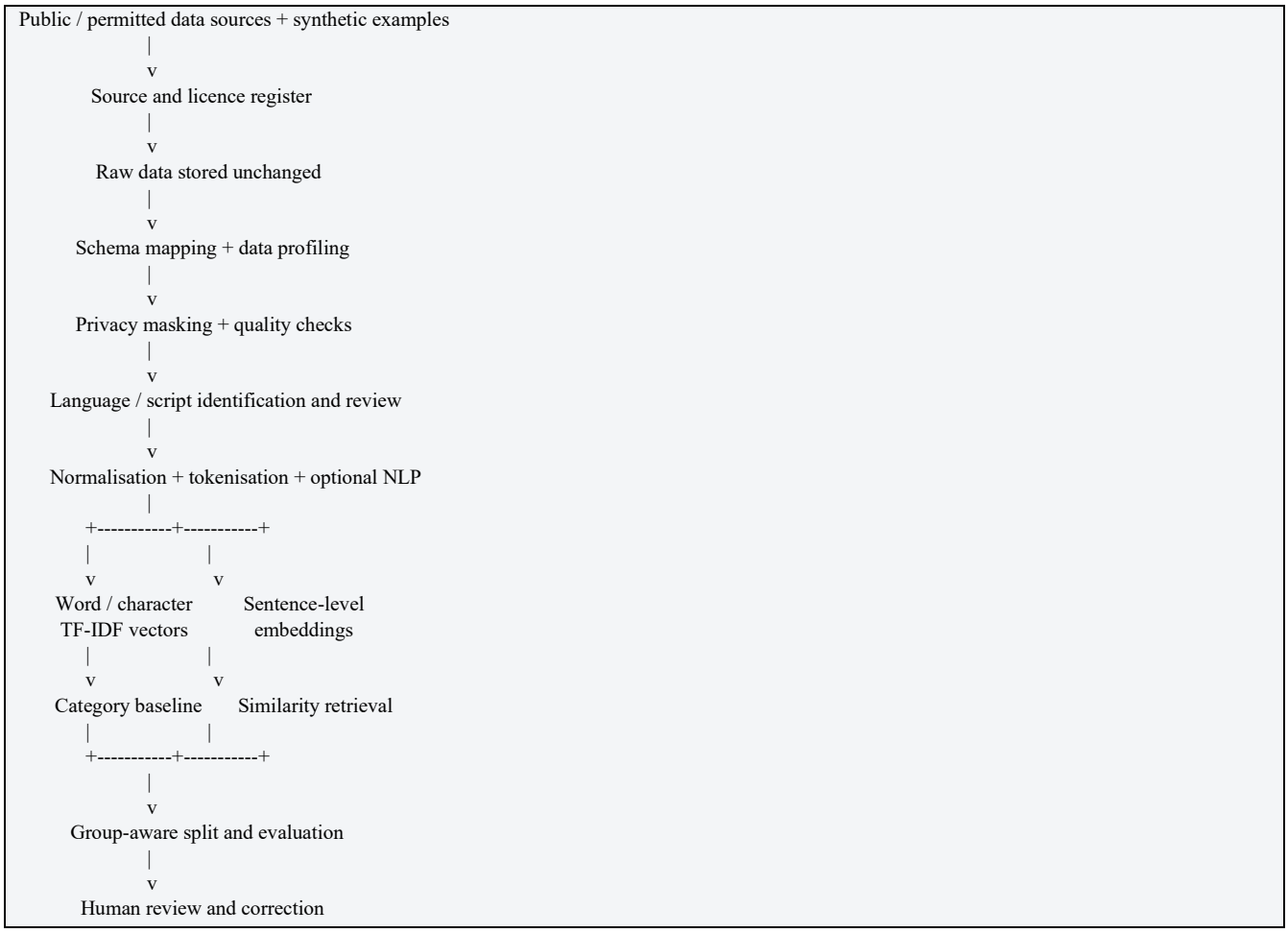
If dimensionality becomes large, use vocabulary limits or minimum document frequency based on validation results. Feature selection methods may be considered, but should be fitted only on training data. Keep a reproducible configuration file recording every selected parameter.

7.5 Class imbalance strategy

If one complaint category has many more examples than another, overall accuracy can hide poor performance on rare categories. Report per-class precision and recall, macro-F1 and a confusion matrix. Use stratified splitting where appropriate, and consider class weights before synthetic oversampling. Any augmentation must be clearly documented and applied only within the training data.

8. End-to-End Workflow and Pseudocode

8.1 Proposed preparation workflow



8.2 Illustrative transformation example

Synthetic input: “Road pe bada hole hai, bikes are falling.”

Stage	Illustrative output
Raw text	Road pe bada hole hai, bikes are falling.
Privacy masking	No personal identifier detected in this example.
Normalisation	road pe bada hole hai, bikes are falling.
Language/script note	Code-mixed Hindi-English; Latin script.
Tokenisation	[road, pe, bada, hole, hai, bikes, are, falling]
Careful stop-word handling	Do not remove meaningful local-language or negation terms without checking the task-specific list.
Word TF-IDF	Sparse weighted values for vocabulary terms and selected word n-grams.
Character TF-IDF	Character fragments that may help match spelling variants.
Sentence embedding	One dense vector for semantic comparison with other complaints.

This example is synthetic and is used only to illustrate the pipeline. Its language label and any category or urgency label would need to be checked against the project annotation guide.

8.3 Pseudocode: collection and cleaning

```
load permitted_dataset
record_source_metadata(dataset_name, url, licence, access_date)

df = map_source_columns_to_project_schema(permitted_dataset)
save_unchanged_copy(df, "data/raw/")

profile = inspect_missing_values_duplicates_languages_and_labels(df)
log_profile(profile)

df = remove_rows_without_usable_text(df)
df["raw_text"] = df["complaint_text"]
df["text_for_processing"] = mask_personal_identifiers(df["raw_text"])
df["text_for_processing"] = unicode_normalise_NFC(df["text_for_processing"])
df["text_for_processing"] = standardise_whitespace(df["text_for_processing"])

df["language_label"], df["script_label"] = estimate_language_and_script(
    df["text_for_processing"]
)
send_low_confidence_language_cases_for_manual_review(df)

save_versioned_clean_copy(df, "data/processed/")
```

8.4 Pseudocode: train-safe TF-IDF workflow

```
# Split by incident / duplicate group before fitting vectorisers
train, valid, test = group_aware_split(
    records,
    group_column="duplicate_group_id"
)

word_vectoriser = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=2,
    sublinear_tf=True
)
X_train_word = word_vectoriser.fit_transform(train.text_for_processing)
X_valid_word = word_vectoriser.transform(valid.text_for_processing)
X_test_word  = word_vectoriser.transform(test.text_for_processing)

# Fit a separate character vectoriser for the character-feature experiment
char_vectoriser = TfidfVectorizer(
    analyzer="char",
    ngram_range=(3, 5),
    min_df=2
)
X_train_char = char_vectoriser.fit_transform(train.text_for_processing)
X_valid_char = char_vectoriser.transform(valid.text_for_processing)
X_test_char  = char_vectoriser.transform(test.text_for_processing)

# Train a baseline classifier and evaluate on held-out data
model.fit(X_train_word, train.category_label)
predictions = model.predict(X_test_word)
report_macro_f1_precision_recall_and_confusion_matrix(
    test.category_label, predictions
)
```

8.5 Pseudocode: sentence-embedding retrieval

```
# Use a multilingual sentence-embedding model after checking its model card
vectors = sentence_encoder.encode(
    complaint_texts,
```

```
    normalise_embeddings=True
)

# For a new complaint, retrieve the most similar candidates
query_vector = sentence_encoder.encode([new_complaint],
                                       normalise_embeddings=True)
scores = cosine_similarity(query_vector, stored_vectors)

candidate_ids = top_k_records(scores, k=5)
show_candidates_with_text_and_scores(candidate_ids, scores)

# Similarity is a review signal, not proof of a duplicate.
reviewer_confirms_or_rejects_candidate_matches()
```

9. Pitfalls, Mitigation and Quality Assurance

Potential pitfall	Why it matters	Mitigation
Using data without checking licence	The project may use or redistribute data beyond permitted terms.	Record licence and terms; use only permitted sources; seek clarification when unclear.
Treating synthetic text as real complaints	Can create misleading claims about real-world coverage.	Mark source_type; report synthetic results separately.
Over-cleaning text	May remove negation, time, urgency or location clues.	Keep raw text; use conservative transformations; inspect before/after examples.
English-only stop-word lists	Can remove the wrong words or fail to process Indian-language text.	Use reviewed language-specific lists; compare with no stop-word removal.
Incorrect language detection	Short or code-mixed messages can be misclassified.	Store confidence/unknown; manually review a sample; allow mixed labels.
Applying English stemming to Indian languages	Can distort words and harm meaning.	Use language-appropriate tools or skip stemming; compare results.
Aggressive transliteration or translation	May erase local phrasing or introduce translation errors.	Preserve original; document optional normalised forms; avoid forced translation.
Duplicate leakage across splits	Inflates evaluation because near-identical reports occur in train and test.	Split by incident or duplicate group before fitting features.
Class and language imbalance	Overall scores may hide poor performance for smaller groups.	Report per-class and per-language metrics; gather more permitted examples.
Privacy leakage	Personal details may be exposed in reports, demos or model inputs.	Mask identifiers, restrict raw data, use synthetic screenshots.
Embedding similarity treated as proof	Same-topic complaints may refer to different incidents.	Tune thresholds on labelled pairs and require human confirmation.
Feature fitting on all records	Vocabulary and IDF can leak information from test data.	Fit vectorisers and feature selectors on training data only.

9.1 Quality assurance checklist

- Every source has a recorded provider, URL, access date and licence/terms status.
- Raw files are unchanged and separated from processed files.
- Each record has a unique project ID and a traceable source identifier.
- Missing text, duplicates and invalid encodings have been measured.
- Privacy masking is tested on representative examples and its errors are reviewed.
- Language and script are represented separately; uncertain cases are not forced into a language.
- Negation, time, quantities and urgency cues survive preprocessing.
- Stop-word lists and stemming/lemmatisation choices are documented and evaluated.
- Train/validation/test separation prevents duplicate-group leakage.
- All vectorisers and feature-selection steps are fitted on training data only.
- Performance is reported separately for languages and complaint categories where sample sizes permit.
- All synthetic examples are labelled as synthetic.

9.2 Example before-and-after audit

For a random sample of records, save a side-by-side audit containing raw text, masked text, normalised text, token list and any transformed form. A reviewer should check whether the cleaned version still communicates the same complaint. If meaning has changed, revise the rule and rerun the pipeline with a new version number.

10. Week 2 Execution Plan and Deliverables

The Week 1 timeline allocates Week 2 to dataset collection, pilot annotation and class-balance analysis, with a version 1 dataset and annotation guide as the expected deliverable. This report expands that plan into the following practical schedule. The hours below are a suggested allocation for the stated 30–35 hour task expectation, not a record of hours already completed.

Activity	Suggested time	Output
Dataset/source discovery and licence review	5 hours	Source register and source-selection rationale.
Download/import and schema mapping	4 hours	Raw-data inventory and mapped dataset.
Data profiling and quality checks	4 hours	Missing-value, duplicate, language and class-balance summary.
Annotation guide and pilot labelling	5 hours	Taxonomy, urgency definitions and pilot-labelled sample.
Privacy masking and text preprocessing design	6 hours	Versioned cleaning pipeline and audit examples.
Feature extraction experiments/design	6 hours	TF-IDF, character n-gram and embedding comparison plan.
Documentation, review and final checks	3 hours	Completed Week 2 report and reproducibility checklist.
Total	33 hours	Week 2 methodology package.

10.1 Expected Week 2 deliverables

- A source register with dataset provenance, licence and language notes.
- A version 1 pilot dataset, if suitable permitted data is available; otherwise a documented schema and synthetic pilot set.
- A complaint category and urgency annotation guide.
- A data-quality profile covering missing values, duplicates, category balance and language/script coverage.
- A documented preprocessing pipeline with before-and-after examples.
- A feature engineering comparison and justified selection for each CivicSift AI task.
- Pseudocode or notebook outline for reproducible preparation and feature extraction.
- A list of limitations and decisions to carry into Week 3.

10.2 Handover to Week 3

Week 3 in the Week 1 schedule focuses on preprocessing, language handling and a TF-IDF baseline. The Week 2 output should therefore make the next stage straightforward: freeze a versioned pilot dataset, confirm the category labels, establish group-aware data splits, and define the TF-IDF configuration to test. Week 3 should then fit the vectoriser on training data, train Logistic Regression and/or Linear SVM, evaluate macro-F1 and per-class performance, and review errors by language.

11. Conclusion

This Week 2 methodology provides a practical and responsible foundation for CivicSift AI. It treats data collection, privacy, annotation, preprocessing and feature engineering as connected stages rather than isolated operations. The proposed approach preserves original complaint text, uses conservative language-aware cleaning, protects meaning-bearing terms, and records source provenance and transformation decisions.

For category classification, word-level and character-level TF-IDF provide an interpretable and computationally practical baseline. For duplicate discovery, multilingual sentence embeddings are a suitable candidate because they compare meaning beyond exact word overlap. IndicBERT or XLM-R may be evaluated later as advanced classification models. All choices must be tested on permitted, representative, group-separated data, and uncertain predictions must remain subject to human review.

The report is a plan for Week 2 work, not evidence that the proposed dataset or models have already been completed. Actual dataset availability, language coverage, licences and annotation quality must be confirmed before implementation.

References and Resources

1. Kakwani, D., Kunchukuttan, A., Golla, S., Gokul, N. C., Bhattacharyya, A., Khapra, M. M., & Kumar, P. (2020). IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages. Findings of EMNLP 2020. https://aclanthology.org/2020.findings-emnlp.1/

2. AI4Bharat. IndicBERT and Indic language resources. https://indicnlp.ai4bharat.org/pages/indic-bert/

3. AI4Bharat. IndicBERT repository and model documentation. https://github.com/AI4Bharat/IndicBERT

4. Conneau, A., et al. (2020). Unsupervised cross-lingual representation learning at scale. Proceedings of ACL 2020. https://aclanthology.org/2020.acl-main.747/

5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP-IJCNLP 2019. https://aclanthology.org/D19-1410/

6. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

7. Scikit-learn documentation. Feature extraction: text feature extraction and TF-IDF vectorisation. https://scikit-learn.org/stable/modules/feature_extraction.html

8. Hugging Face. Transformers documentation: multilingual models and tokenizers. https://huggingface.co/docs/transformers/

9. NYC Open Data. Official open-data portal (candidate source for exploring civic-service request data; check dataset-specific terms and fields before use). https://opendata.cityofnewyork.us/

10. COMI-LINGUA. Expert-annotated Hindi-English code-mixed NLP resource. Findings of EMNLP 2025. https://lingo-iitgn.github.io/projects/code-mixing.html

Appendix A. Sample synthetic complaint records

ID	Complaint text (synthetic)	Language/script	Possible category
SYN-001	There is water leakage near the school since morning.	English / Latin	Water
SYN-002	Street light kaam nahi kar rahi hai.	Hindi-English mixed / Latin	Electricity
SYN-003	ರಸ್ತೆಯಲ್ಲಿ ದೊಡ್ಡ ಗುಂಡಿ ಇದೆ.	Kannada / Kannada	Roads
SYN-004	Road pe bada hole hai, bikes are falling.	Hindi-English mixed / Latin	Roads
SYN-005	குப்பை?	Not in current language scope; illustrative out-of-scope case	Other / review

All records in this appendix are synthetic illustrations. They are not real citizen reports, verified labels or model predictions. The out-of-scope sample demonstrates that language scope must be enforced and documented.

Appendix B. Suggested project folder structure

```
civicsift-ai/
├── data/
│   ├── raw/              # original permitted files; restricted access
│   ├── interim/          # mapped / profiled data
│   └── processed/        # versioned masked and cleaned data
├── notebooks/
│   ├── 01_data_audit.ipynb
│   └── 02_preprocessing_features.ipynb
├── src/
│   ├── collect_data.py
│   ├── data_validation.py
│   ├── privacy_masking.py
│   ├── preprocessing.py
│   ├── feature_extraction.py
│   └── evaluation.py
├── reports/
│   ├── source_register.csv
│   └── preprocessing_log.md
├── requirements.txt
└── README.md
```

Appendix C. Suggested preprocessing log fields

Log field	Purpose
pipeline_version	Identify the exact transformation version.
run_date	Record when the pipeline was executed.
input_file / checksum	Identify the raw source version.
records_in / records_out	Measure the effect of filtering.
masked_identifier_counts	Record counts of detected/masked identifier types.
normalisation_rules	List Unicode and whitespace rules applied.
language_review_summary	Record automatic estimates and manual corrections.
tokeniser / stopword_list_version	Make language processing reproducible.
stemming_or_lemma_tool	Record tool, language and configuration, or state not applied.
feature_configuration	Record n-gram ranges and vectoriser parameters.
reviewer_notes	Document errors, exceptions and unresolved issues.

Appendix D. Submission checklist

- Confirm the student, internship and supervisor details on the cover page.
- Confirm the exact project title and language scope with the supervisor.
- Verify source licences, dataset terms and model licences before downloading or using data.
- Do not state that data was collected or models were trained unless that work has actually been completed.
- Keep synthetic examples visibly labelled.
- Keep proposed targets and future work separate from achieved results.
- Check spelling, tables, page breaks and references before submission.